C'est une excellente question d'architecture. Explorer différentes sources est exactement ce qu'il faut faire lors d'un PoC !

Je vais te donner le code, mais je dois d'abord te faire une révélation très franche : **Spark Structured Streaming n'a pas été conçu pour lire directement des API REST.** Spark adore qu'on lui "pousse" de la donnée (comme Kafka) ou qu'on lui dépose des fichiers (comme S3). Une API REST, c'est l'inverse : il faut aller la "tirer" (faire des requêtes GET en boucle). Spark n'a pas de connecteur natif pour ça.

Cependant, il existe un "hack" très connu dans l'industrie pour contourner ça : on utilise une source factice appelée rate (qui génère un "tic-tac" régulier) et on attache une fonction personnalisée (UDF) qui va appeler l'API à chaque "tic".

Voici à quoi ressemblerait ton fichier streaming_app.py pour tester cette viabilité :

Le Code (Spark Streaming via API REST)

Python

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import col, udf

from pyspark.sql.types import StringType

import requests

import json


# 1. Initialisation de la session (avec notre fameux anti-logs !)

spark = SparkSession.builder \

    .appName("poc-streaming-api") \

    .config("spark.eventLog.enabled", "false") \

    .getOrCreate()


spark.sparkContext.setLogLevel("WARN")


# 2. La fonction Python qui va appeler ton API REST

def fetch_from_api(tick_value):

    # Remplace par l'URL de ton API de test
```

```
    api_url = "https://jsonplaceholder.typicode.com/posts/1"
    try:
        response = requests.get(api_url, timeout=5)
        if response.status_code == 200:
            return response.text # Retourne le JSON sous forme de texte
        else:
            return f'{{"error": "HTTP {response.status_code}"}}'
    except Exception as e:
        return f'{{"error": "{str(e)}"}}'

# On transforme cette fonction Python en UDF (User Defined Function) Spark
api_udf = udf(fetch_from_api, StringType())

# 3. La Source "Rate" (Le métronome)
# Génère 1 ligne par minute (pour ne pas spammer l'API)
# Attention: si tu mets rowsPerSecond=10, Spark fera 10 requêtes API par seconde !
metronome_df = spark.readStream \
    .format("rate") \
    .option("rowsPerSecond", 1) \
    .load()

# 4. Le Traitement : À chaque tic du métronome, on appelle l'API
api_data_df = metronome_df \
    .withColumn("json_payload", api_udf(col("value"))) \
    .select("timestamp", "json_payload")

# 5. La Destination (Console pour le test, ou S3 comme avant)
print("### DÉMARRAGE DU STREAMING API ###")
```

```
query = api_data_df.writeStream \
    .format("console") \
    .outputMode("append") \
    .option("truncate", "false") \
    .start()

query.awaitTermination()
```

Comparaison : Est-ce une option viable en production ?

Pour être tout à fait candide : **Non, lire une API REST directement depuis Spark Streaming est une mauvaise pratique en production.**

Voici la comparaison avec ce que tu as déjà fait (S3) :

Critère	**Fichiers sur S3 (Ton code actuel)** ⏳	**API REST directe dans Spark** ❌
Fiabilité	Parfaite. Si Spark plante, les fichiers attendent sagement sur S3.	Mauvaise. Si l'API ne répond pas ou a un hoquet, le job Spark plante.
Gestion de la charge	Spark lit à son rythme.	**Risque de DDoS**. Si tu ajoutes 10 Executors, Spark va faire 10 fois plus de requêtes simultanées et peut faire crasher l'API cible.
Perte de données	Aucune (Checkpoints natifs très robustes).	Élevée. Si l'API renvoie des données en temps réel et que Spark redémarre, la donnée générée pendant la panne est perdue à jamais.

La vraie méthode de l'industrie pour les API

Si ton entreprise veut absolument consommer une API REST en temps réel, on n'utilise jamais Spark pour faire l'appel HTTP. On utilise une architecture en deux temps :

1. Un petit outil léger (comme un script Python basique, Apache NiFi, ou Logstash) interroge l'API en boucle et envoie les résultats dans **Apache Kafka** (un bus de messages).
2. Ton job **Spark Streaming écoute Kafka** (qui est fait pour ça, au lieu du readStream.format("csv"), on fait un readStream.format("kafka")).

Que veux-tu faire ? Veux-tu que l'on intègre ce script "Hack API" dans ton submit_streaming.sh pour que tu puisses voir le résultat de tes propres yeux sur le cluster, ou préfères-tu qu'on explore la façon de lire tes fichiers Parquet fraîchement créés sur ton bucket S3 ?

C'est la transition parfaite ! Tu mets exactement le doigt sur la façon dont les architectures modernes (Event-Driven) sont construites en production.

Plutôt que de dire à Spark *"Regarde ce dossier S3 toutes les secondes pour voir s'il y a un nouveau fichier"* ou *"Appelle cette API en boucle"*, on utilise un système d'**Événements Cloud (Cloud Events)**. Dès qu'il se passe quelque chose (un client clique, une base de données est mise à jour, une API reçoit un POST), un événement est envoyé dans un "tuyau", et Spark écoute ce tuyau en temps réel.

Puisque tu es sur un cluster Kubernetes et que tu utilises IBM Cloud, voici les 3 grandes options viables pour ton pipeline, de la plus standard à la plus spécifique.

Option 1 : Le Roi de l'Industrie (Apache Kafka / IBM Event Streams) 👑

C'est l'option par défaut pour 95 % des projets de streaming Big Data dans le monde.

Kafka est un "bus de messages". Les applications (ou tes API) envoient leurs événements (souvent au format JSON) dans un "Topic" Kafka. Spark est connecté à ce Topic et consomme les messages à la milliseconde près.

- **Sur IBM Cloud :** Le service s'appelle **IBM Event Streams** (c'est du Kafka managé par IBM).
- **Pourquoi c'est génial pour Spark :** Spark Structured Streaming a été littéralement pensé pour Kafka. Il gère nativement les coupures réseau, garantit qu'aucun message n'est lu deux fois (Exactly-Once semantics), et gère les pics de charge (si tu reçois 1 million d'événements à la seconde, Kafka les garde au chaud pendant que Spark les traite à son rythme).
- **Le code :** Au lieu de format("csv"), tu utilises format("kafka").

Option 2 : Les coursiers légers (RabbitMQ / MQTT / ActiveMQ) 🐇

Si tes événements viennent d'objets connectés (IoT) ou de petites applications internes qui génèrent peu de volume, un broker de messages classique comme RabbitMQ est souvent utilisé.

- **Le principe :** Similaire à Kafka, mais conçu pour acheminer des messages précis plutôt que de stocker des flux massifs de Big Data.
- **Le hic avec Spark :** Spark n'a pas de connecteur readStream officiel natif pour RabbitMQ dans sa dernière version. Il faut souvent installer des plugins externes (JARs) développés par la communauté, ce qui peut rendre la configuration sur Kubernetes un peu plus capricieuse que Kafka.

Option 3 : Le standard Kubernetes (Knative Eventing / CloudEvents) ☸️

Puisque tu es sur Kubernetes, tu entendras peut-être parler de "Knative" et du format officiel "CloudEvents" (de la Cloud Native Computing Foundation).

- **Le principe :** Kubernetes lui-même devient le routeur. Une application émet un événement au format standard CloudEvent, et Kubernetes "réveille" ou notifie le bon pod pour le traiter.
- **La réalité pour ton PoC :** C'est ultra-puissant pour déclencher des petits scripts (Serverless/Functions), mais **Spark Streaming ne fonctionne pas bien comme ça**. Spark a besoin d'être allumé en permanence et d'avoir un "tuyau" continu dans lequel puiser. Généralement, si on utilise Knative, on configure Knative pour qu'il pousse ses CloudEvents... dans Kafka ! Et Spark lit Kafka.

Bilan : Que choisir pour ton architecture ?

Si ton objectif est de faire du traitement de données en continu, robuste, et sans risquer de surcharger tes API sources, **Kafka (ou IBM Event Streams) est l'option que tu dois privilégier.**

C'est le mariage parfait :

1. Ton API (ou l'outil de ton choix) envoie un JSON dans Kafka.
2. Ton pod Spark Kubernetes lit Kafka en temps réel, transforme le JSON, et l'écrit en Parquet sur ton bucket IBM COS bu002i004226.

Comment veux-tu procéder ? Souhaites-tu que l'on regarde à quoi ressemblerait ton script Python streaming_app.py s'il devait se brancher sur un flux d'événements Kafka au lieu de ton dossier S3 actuel ?